

# Claude Code 超入門ガイド

DB設計 / フォルダ構造 / 技術アーキテクチャ

STEP 2 / 3

## 1. 技術スタック（選定理由付き）

| レイヤー    | 技術                                | 選定理由              |
|---------|-----------------------------------|-------------------|
| フロントエンド | Next.js 14 (App Router)           | SSR/SSG両対応・SEO強い  |
| 言語      | TypeScript (strict)               | バグ防止・補完が効く        |
| スタイル    | Tailwind CSS                      | Apple品質UIが速く作れる   |
| コンポーネント | shadcn/ui                         | 美しいUIを即使用可能       |
| DB      | Supabase (PostgreSQL)             | 認証・リアルタイム・無料枠大    |
| ORM     | Prisma                            | 型安全なDB操作          |
| AI機能    | Anthropic SDK (claude-3-5-sonnet) | 質問回答・エラー診断        |
| 検索      | Fuse.js                           | 日本語対応のあいまい検索      |
| 状態管理    | Zustand                           | シンプルで軽量           |
| 認証      | Supabase Auth (Google/GitHub)     | 設定が簡単・無料          |
| デプロイ    | Vercel                            | Next.jsと相性最高・無料枠大 |
| 分析      | Vercel Analytics                  | ページごとの滞在時間確認      |

## 2. フォルダ構造（完全版）

claude-guide/

```
|
|├── src/
|   |├── app/                                ← Next.js App Router
|   |   |├── (public)/                      ← 認証不要ページ
|   |   |   |├── page.tsx                  ← LP (ランディングページ)
|   |   |   |├── start/                    ← 初心者一本道モード
|   |   |   |   |├── page.tsx              ← モード選択
|   |   |   |   |├── [step]/              ← 各ステップ動的ルート
|   |   |   |   |   |├── page.tsx
|   |   |   |   |├── learn/               ← ツリーナビ学習
|   |   |   |   |   |├── page.tsx          ← ツリー一覧
|   |   |   |   |   |├── [...slug]/        ← 動的ルート
|   |   |   |   |   |   |├── page.tsx
|   |   |   |   |├── check/page.tsx        ← 環境診断
|   |   |   |   |├── error/page.tsx        ← エラー診断
|   |   |   |   |├── translate/page.tsx    ← 英語翻訳辞書
|   |   |   |   |├── prompts/page.tsx     ← プロンプト生成器
|   |   |   |
|   |   |   |├── (auth)/                  ← 認証後ページ
|   |   |   |   |├── progress/page.tsx     ← 進捗管理
|   |   |   |
|   |   |   |├── api/                    ← APIルート
|   |   |   |   |├── ai/
|   |   |   |   |   |├── question/route.ts ← AI質問機能
|   |   |   |   |   |├── error/route.ts   ← エラー診断AI
|   |   |   |   |├── progress/route.ts    ← 進捗保存
|   |   |   |   |├── prompts/route.ts     ← プロンプト生成
|   |   |   |
|   |   |   |├── components/
|   |   |   |   |├── layout/
|   |   |   |   |   |├── Header.tsx
|   |   |   |   |   |├── TreeNav.tsx      ← 左サイドバーツリー
|   |   |   |   |   |├── ProgressBar.tsx ← 上部進捗バー
|   |   |   |   |
|   |   |   |   |├── learn/
|   |   |   |   |   |├── StepCard.tsx     ← 各ステップカード (再利用コンポーネント)
|   |   |   |   |   |├── CommandBlock.tsx ← コピー可能コードブロック
|   |   |   |   |   |├── TranslationCard.tsx ← 英語→日本語翻訳カード
|   |   |   |   |   |├── ErrorChecker.tsx ← エラー診断UI
|   |   |   |   |   |├── AskAI.tsx       ← AIに聞くボタン&モーダル
|   |   |   |   |
|   |   |   |   |├── progress/
|   |   |   |   |   |├── ChecklistItem.tsx ← チェックリスト1項目
|   |   |   |   |   |├── StreakBadge.tsx  ← 連続学習バッジ
|   |   |   |   |
|   |   |   |   |├── ui/                  ← shadcn/ui コンポーネント
|   |   |   |
|   |   |   |├── lib/
|   |   |   |   |├── supabase/
|   |   |   |   |   |├── client.ts        ← Supabaseクライアント
```

```

├── server.ts          ← サーバースイドクライアント
├── ai/
│   ├── question.ts   ← AI質問ロジック
│   └── error-analyzer.ts ← エラー分析ロジック
├── content/
│   ├── loader.ts      ← MDXコンテンツ読み込み
│   └── utils/
│       ├── error-patterns.ts ← エラーパターン辞書
│       └── progress.ts     ← 進捗計算ユーティリティ
├── content/           ← MDXコンテンツ（Gitで管理）
│   ├── steps/
│   │   ├── install-windows.mdx
│   │   ├── install-mac.mdx
│   │   ├── setup-apikey.mdx
│   │   └── ...
│   ├── projects/
│   │   ├── webapp.mdx
│   │   ├── stock-app.mdx
│   │   └── ...
│   └── translations/
│       └── ui-dictionary.json ← 英語UI辞書
├── types/
│   └── index.ts        ← 型定義
├── prisma/
│   └── schema.prisma
├── public/
│   ├── screenshots/   ← 操作画面のスクリーンショット
│   └── icons/
└── ...設定ファイル群

```

### 3. データベース設計（Prisma Schema）

// — ユーザー

```

model User {
  id          String    @id @default(cuid())
  email       String    @unique
  name        String?
  avatarUrl   String?
  os          String?   // "windows" | "mac" | "linux"
  experienceLevel String? // "zero" | "beginner" | "intermediate"
  createdAt   DateTime  @default(now())

  progress    Progress[]
  questions   AiQuestion[]
}

```

```

    @@map("users")
}

// —— 進捗管理


---


model Progress {
    id          String   @id @default(cuid())
    userId      String
    stepId      String   // "install-windows-node" など
    status      String   // "not_started" | "in_progress" | "done"
    completedAt DateTime?
    createdAt   DateTime @default(now())
    updatedAt   DateTime @updatedAt

    user        User @relation(fields: [userId], references: [id])

    @@unique([userId, stepId])
    @@map("progress")
}

// —— AI質問履歴


---


model AiQuestion {
    id          String   @id @default(cuid())
    userId      String?  // 未ログインも可
    stepId      String?  // どのステップで質問したか
    question    String
    answer      String
    helpful     Boolean? // 「役に立った」 ボタンの結果
    createdAt   DateTime @default(now())

    user        User? @relation(fields: [userId], references: [id])

    @@map("ai_questions")
}

// —— エラー診断ログ


---


model ErrorLog {
    id          String   @id @default(cuid())
    errorText   String   // ユーザーが貼ったエラー
    category    String   // "path" | "permission" | "network" | "api_key" | "other"
    solution    String   // 提示した解決策
    resolved    Boolean? // 解決できたか
    createdAt   DateTime @default(now())

    @@map("error_logs")
}

```

## 4. コンテンツ管理方式

MDXファイルでコンテンツを管理。コードとコンテンツを分離することで、**Claude Code**でコンテンツを量産できる設計。

```
---
id: "install-windows-node"
title: "Node.jsをインストールする"
description: "Claude Codeを動かすための土台を入れます"
os: ["windows"]
estimatedMinutes: 5
order: 1
---

import { CommandBlock } from '@components/learn/CommandBlock'
import { TranslationCard } from '@components/learn/TranslationCard'

## なぜ必要か

Claude Codeを動かすための「土台」です。...

## 操作手順

1. [公式サイト](https://nodejs.org)を開く
2. 緑色の **「LTS」** ボタンをクリック

## 確認コマンド

<CommandBlock command="node --version" expected="v20.x.x" />

## よくある失敗

<ErrorPattern
  error="'node' は内部コマンドまたは外部コマンドとして認識されていません"
  cause="パス（PATH）が設定されていません"
  solution="パソコンを再起動してから試してください"
/>
```

## 5. AI機能アーキテクチャ（Rate Limit対策付き）

ユーザーが「AIに聞く」を押す

↓

① キャッシュチェック（同じ質問が過去にあるか？）  
ヒット → キャッシュから回答を即返す（APIコール0）

↓

② なければ Anthropic API に送信  
モデル: claude-3-5-haiku（高速・安価）  
max\_tokens: 500（初心者向けに短く明確に）

↓

③ 回答をDBにキャッシュ保存

↓

④ ユーザーに返す

Rate Limit対策:

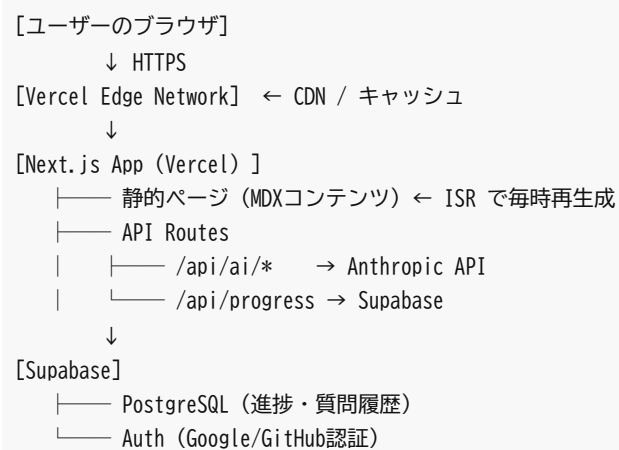
- ユーザーごとに1分5回まで制限
- 同一エラー文はキャッシュ返却
- 「役に立った/立たなかった」で質問品質改善

## 6. 環境診断システム

```
// check/ ページで実行するチェック項目

const CHECKS = [
  {
    id: "node",
    label: "Node.js",
    description: "Claude Codeの土台",
    command: "node --version",
    successPattern: /v\d+¥.¥d+/,
    failGuide: "/learn/install/windows#node"
  },
  {
    id: "claude",
    label: "Claude Code",
    description: "AIアシスタント",
    command: "claude --version",
    successPattern: /¥d+¥.¥d+/,
    failGuide: "/learn/install/windows#claude"
  },
  {
    id: "git",
    label: "Git",
    description: "変更履歴の管理",
    command: "git --version",
    successPattern: /git version/,
    failGuide: "/learn/install/windows#git"
  },
  {
    id: "apikey",
    label: "APIキー",
    description: "Claudeへの合言葉",
    // ブラウザからは実行不可 → ユーザーに手動確認を促す
    type: "manual",
    failGuide: "/learn/setup/apikey"
  }
]
```

## 7. デプロイ構成



コスト試算（月間1,000ユーザー想定）：

| サービス                   | 費用        |
|------------------------|-----------|
| Vercel（Hobbyプラン）       | ¥0        |
| Supabase（Freeプラン）      | ¥0        |
| Anthropic API（月5,000回） | ~¥2,000   |
| 合計                     | ~¥2,000/月 |